



Sandya Mannarswamy

Welcome to another installment of CodeSport. This month, we will continue our discussion on deadlocks in multi-threaded applications. We will also discuss the issue of false sharing in multi-threaded applications.

Thanks to all the readers who sent in their responses to the problems we discussed in the previous issue. In last month's takeaway problem, I had given a small snippet of multi-threaded code and asked you to find out what could have been the potential bug hiding in it. Congratulations to our readers Nilesh Govande, Vivek Goel, Arivindu Bhardwaj, Abel Sajaykumar and Mrigendra Nagar for getting the correct answer. As pointed out by these readers, the code snippet had a potential deadlock situation. Here is the buggy code snippet from the takeaway problem:

```
void *counter_func(void *)  
  
int count_var;  
mutex_t count_var_lock;  
  
main()  
{  
    char str[80];  
    pthread_t child_thread_id;  
  
    pthread_create(&child_thread_id, NULL, counter_func);  
  
    while(1)  
    {  
  
        //read the next string  
        scanf ("%s", str);  
        pthread_suspend(child_thread_id);
```

```
        mutex_lock(&count_var_lock);  
        printf("count value is = %d\n", count_var);  
        mutex_unlock(&count_var_lock);  
  
        pthread_continue(child_thread_id);  
    }  
  
    return(0);  
}  
  
void *counter_func(void *arg)  
{  
    int i;  
  
    while (1)  
    {  
        printf("incrementing the counter value\n");  
        mutex_lock(&count_var_lock);  
        count_var++;  
  
        //do nothing  
        for (int i=0; i<MAXCNT; i++);  
  
        mutex_unlock(&count_var_lock);  
  
        //do nothing  
        for (int i=0; i<MAXCNT; i++);  
    }  
  
    return((void *)0);  
}
```

As these readers correctly pointed out, the following is the deadlock condition: *child thread(counter_func)* has locked the *count_*